In [ ]:
```
# 2.1 Логистическая регрессия.
```

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

In [ ]:
```
## 1. Загрузка и обработка датасета.
Дататсет, его загрузка и обработка как в задании 1.1.
```

In [3]:
```
df = pd.read_csv('C:/Users/Elizaveta/Downloads/magic_gamma/telescope_data.csv')
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
df.head(5)
df.info()
print("размер", df.shape)
print("названия столбцов", list(df.columns))
print("уникальные значения в class", df['class'].unique())
print("\nраспределение классов")
print(df['class'].value_counts())
print("доли классов")
print(df['class'].value_counts(normalize='True'))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19020 entries, 0 to 19019
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype  
---  ------      --------------  -----  
 0   Unnamed: 0  19020 non-null  int64  
 1   fLength     19020 non-null  float64
 2   fWidth      19020 non-null  float64
 3   fSize       19020 non-null  float64
 4   fConc       19020 non-null  float64
 5   fConc1      19020 non-null  float64
 6   fAsym       19020 non-null  float64
 7   fM3Long     19020 non-null  float64
 8   fM3Trans    19020 non-null  float64
 9   fAlpha      19020 non-null  float64
 10  fDist       19020 non-null  float64
 11  class       19020 non-null  object 
dtypes: float64(10), int64(1), object(1)
memory usage: 1.7+ MB
размер (19020, 12)
названия столбцов ['Unnamed: 0', 'fLength', 'fWidth', 'fSize', 'fConc', 'fConc1',
'fAsym', 'fM3Long', 'fM3Trans', 'fAlpha', 'fDist', 'class']
уникальные значения в class ['g' 'h']

распределение классов
class
g    12332
h     6688
Name: count, dtype: int64
доли классов
class
g    0.64837
h    0.35163
Name: proportion, dtype: float64
```

In [4]:
```
df['class']=df['class'].replace({'g': 1, 'h': 0})
print("first 10 values", df['class'].head(10))
print("unique values", df['class'].unique())
print("value counts", df['class'].value_counts())
print("type of data in class", df['class'].dtype)
print("first value", repr(df['class'].iloc[0]))
print("NaN", df['class'].isna().sum())
df=df.drop('Unnamed: 0', axis=1)
```

```
first 10 values 0    1
1    1
2    1
3    1
4    1
5    1
6    1
7    1
8    1
9    1
Name: class, dtype: int64
unique values [1 0]
value counts class
1    12332
0     6688
Name: count, dtype: int64
type of data in class int64
first value np.int64(1)
NaN 0
```

```
C:\Users\Elizaveta\AppData\Local\Temp\ipykernel_18956\2620301820.py:1: FutureWarn
ing: Downcasting behavior in `replace` is deprecated and will be removed in a fut
ure version. To retain the old behavior, explicitly call `result.infer_objects(co
py=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silen
t_downcasting', True)`
  df['class']=df['class'].replace({'g': 1, 'h': 0})
```

In [5]:
```
X=df.drop('class', axis=1)
Y=df['class']
print("size of X", {X.shape})
print("size of Y", {Y.shape})
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y,
    test_size=0.3,
    random_state = 42,
    stratify = Y)
print(f"size of X_train", {X_train.shape})
print(f"size of X_test", {X_test.shape})
print(f"size of Y_train", {Y_train.shape})
print(f"size of Y_test", {Y_test.shape})
print("balance of classes:")
print(f"start data: 1={Y.mean():.3f}, 0={1-Y.mean():.3f}")
print(f"train data: 1={Y_train.mean():.3f}, 0={1-Y_train.mean():.3f}")
print(f"start data: 1={Y_test.mean():.3f}, 0={1-Y_test.mean():.3f}")
```

```
size of X {(19020, 10)}
size of Y {(19020,)}
size of X_train {(13314, 10)}
size of X_test {(5706, 10)}
size of Y_train {(13314,)}
size of Y_test {(5706,)}
balance of classes:
start data: 1=0.648, 0=0.352
train data: 1=0.648, 0=0.352
start data: 1=0.648, 0=0.352
```

In [ ]:
```
# 2. Создание бейзлайна.
Обучение модели LogisticRegression из библиотеки sklearn.
F1 учитывает оба класса и их ошибки, показывает точность (сколько из отнесённых
и полноту (сколько найдено настоящих объектов класса) классификации. В случае, е
при высоком Accuracy, F1 покажет это.
```

```
Accuracy показывает, какую долю объектов модель классифицировала правильно. В да
поэтому эта метрика даст адекватную оценку.
Обучающая выборка (train) используется для обучения модели.
Тестовая выборка (test) используется для оценки качества обученной модели.
Результат на бейзлайне: f1=0.8420, accuracy=0.7813.
Получены достаточно хорошие результаты, но есть потенциал для улучшения. Эти дан
отталкиваться в сторону улучшений, показывающей, чего можно достичь минимальными
```

In [30]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, accuracy_score
log_reg_baseline = LogisticRegression(penalty='l2', solver='lbfgs', max_iter=100
log_reg_baseline.fit(X_train, Y_train)
Y_train_prediction = log_reg_baseline.predict(X_train)
Y_test_prediction = log_reg_baseline.predict(X_test)
F1_baseline = f1_score(Y_test, Y_test_prediction)
accuracy_baseline = accuracy_score(Y_test, Y_test_prediction)
print("\nРезультаты Sklearn:")
print(f"F1: {F1_baseline:.4f}")
print(f"accuracy: {accuracy_baseline:.4f}")
```

```
Результаты Sklearn:
F1: 0.8420
accuracy: 0.7813
```

In [ ]:
```
##3. Улучшение бейзлайна.
Гипотеза 1: Масштабирование.
Логистическая регрессия чувствительна к масштабу признаков. Применяем StandardSc
в результате улучшения по метрикам не произошло. Но обучение должно становиться
```

In [9]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
log_reg_scaled = LogisticRegression(penalty='l2', solver='lbfgs', max_iter=1000,
log_reg_scaled.fit(X_train_scaled, Y_train)
Y_train_scaled_prediction = log_reg_scaled.predict(X_train_scaled)
Y_test_scaled_prediction = log_reg_scaled.predict(X_test_scaled)
F1_scaled = f1_score(Y_test, Y_test_scaled_prediction)
accuracy_scaled= accuracy_score(Y_test, Y_test_scaled_prediction)
print(f"F1: {F1_scaled:.4f}")
print(f"accuracy: {accuracy_scaled:.4f}")
```

```
F1: 0.8418
accuracy: 0.7813
```

In [ ]:
```
Гипотеза 2: Оптимизация гиперпараметров.
В логистической регрессии ключевой гиперпараметр - коэффициент регуляризации C,
переобучения, так и недообучения. Перебираются параметры от слабой регуляризации
{'C': 0.1}
0.7952 (улучшение accuracy на 1.7%).
```

In [10]:
```
from sklearn.model_selection import GridSearchCV
param_grid = {'C':[0.01, 0.1, 1, 10, 100]}
scoring = {'Accuracy': 'accuracy', 'F1': 'f1'}
grid_search = GridSearchCV(LogisticRegression(penalty='l2', solver='lbfgs', max_
                           param_grid, cv=5, scoring=scoring, refit='Accuracy')
grid_search.fit(X_train_scaled, Y_train)
best_params = grid_search.best_params_
best_score = grid_search.best_score_
```

```
print(best_params)
print(best_score)
```

```
{'C': 0.1}
0.7952530898279649
```

In [ ]:
```
Извлекаем лучшую модель и применяем ее к тестовой выборке для оценки качества.
F1: 0.8414
accuracy: 0.7808
К улучшению результатов не привело, модель устойчива и не требует настройки опти
```

In [13]:
```
best_log_reg = grid_search.best_estimator_
Y_test_pred = best_log_reg.predict(X_test_scaled)
f1 = f1_score(Y_test, Y_test_pred)
accuracy = accuracy_score(Y_test, Y_test_pred)
print(f"F1: {f1:.4f}")
print(f"accuracy: {accuracy:.4f}")
```

```
F1: 0.8414
accuracy: 0.7808
```

In [ ]:
```
Гипотеза 3: Полиномиальные признаки.
Полиномиальные признаки позволяют учитывать нелинейные взаимодействия между исхо
Применяется масштабирование, т.к. полиномиальные члены могут быть очень большими
частотам. Результаты:
F1: 0.8808
accuracy: 0.8465
Улучшилось качество модели по сравнению с обычной логистической регрессией. Это
классов улучшила чувствительность к меньшему классу.
```

In [17]:
```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)
poly.fit(X_train)
X_train_poly = poly.transform(X_train)
X_test_poly = poly.transform(X_test)
scaler = StandardScaler()
X_train_poly_scaled = scaler.fit_transform(X_train_poly)
X_test_poly_scaled = scaler.transform(X_test_poly)
log_reg_poly_scaled = LogisticRegression(penalty='l2', class_weight='balanced',
log_reg_poly_scaled.fit(X_train_poly_scaled, Y_train)
Y_train_poly_scaled_prediction = log_reg_poly_scaled.predict(X_train_poly_scaled
Y_test_poly_scaled_prediction = log_reg_poly_scaled.predict(X_test_poly_scaled)
F1_poly_scaled = f1_score(Y_test, Y_test_poly_scaled_prediction)
accuracy_poly_scaled= accuracy_score(Y_test, Y_test_poly_scaled_prediction)
print(f"F1: {F1_poly_scaled:.4f}")
print(f"accuracy: {accuracy_poly_scaled:.4f}")
```

```
F1: 0.8808
accuracy: 0.8465
```

In [ ]:
```
Гипотеза 4: Подбор гиперпараметров для полиномиальных признаков.
После того, как убедились, что полиномиальные признаки улучшают качество модели,
Перебираются все комбинации из 5 значений C и 2 вариантов class_weight. Использу
{'C': 100, 'class_weight': None}
accuracy=0.8605
Оптимальна слабая регуляризация и отказ от балансировки классов.
```

In [18]:
```
param_grid = {'C':[0.01, 0.1, 1, 10, 100], 'class_weight':[None, 'balanced']}
grid_search_poly = GridSearchCV(LogisticRegression(penalty='l2', solver='lbfgs',
                           param_grid, cv=5, scoring='accuracy', refit=True)
```

```
grid_search_poly.fit(X_train_poly_scaled, Y_train)
best_params_poly = grid_search_poly.best_params_
best_score_poly = grid_search_poly.best_score_
print(best_params_poly)
print(best_score_poly)
```

```
{'C': 100, 'class_weight': None}
0.8605976436984776
```

In [ ]:
```
## 4. Имплементация алгоритма машинного обучения.
Был создан собственный алгоритм логистической регрессии MyLogReg с использованием
инициализацию параметров, сигмоидную функцию, вычисление логистической функции п
Результаты:
F1: 0.7939
accuracy: 0.6717
Хуже, чем базовая модель, требуется улучшение.
```

In [24]:
```
class MyLogReg:
    def __init__(self, learning_rate=0.01, max_iter=1000, alpha=0.0):
        self.lr = learning_rate
        self.n_iter = max_iter
        self.alpha = alpha
        self.weights = None
        self.bias = None
        self.loss_history = []
    def sigmoid(self, z):
        z = np.clip(z, -500, 500)
        return 1/(1+np.exp(-z))
    def fit(self, X, Y):
        m, n = X.shape
        self.weights = np.zeros(n)
        self.bias = 0.0
        for i in range(self.n_iter):
            model = X@self.weights+self.bias
            Y_prediction = self.sigmoid(model)
            loss = -np.mean(Y*np.log(Y_prediction+1e-15)+(1-Y)*np.log(1-Y_predic
            self.loss_history.append(loss)
            dw = (1/m)*X.T@(Y_prediction-Y)
            db = (1/m)*np.sum(Y_prediction-Y)
            if self.alpha>0:
                dw+=self.alpha*self.weights
            self.weights -= self.lr*dw
            self.bias -= self.lr*db
            if i%100 ==0:
                print(f"iter {i}, loss = {loss:.6f}")
        return self
    def predict_proba(self, X):
        model = X@self.weights+self.bias
        return self.sigmoid(model)
    def predict(self, X, threshold=0.5):
        proba = self.predict_proba(X)
        return (proba>=threshold).astype(int)

my_log_reg = MyLogReg(learning_rate=0.01, max_iter=1000, alpha=0.0)
my_log_reg.fit(X_train, Y_train)
Y_prediction_my = my_log_reg.predict(X_test)
f1_my_log = f1_score(Y_test, Y_prediction_my)
accuracy_my_log = accuracy_score(Y_test, Y_prediction_my)
print(f" F1: {f1_my_log:.4f}")
print(f" accuracy: {accuracy_my_log:.4f}")
```

```
iter 0, loss = 0.693147
iter 100, loss = 12.527268
iter 200, loss = 10.856811
iter 300, loss = 6.228574
iter 400, loss = 10.465181
iter 500, loss = 8.540146
iter 600, loss = 11.337971
iter 700, loss = 8.940309
iter 800, loss = 10.950148
iter 900, loss = 11.386846
 F1: 0.7939
 accuracy: 0.6717
```

In [ ]:
```
После корректировки гиперпараметров наблюдается улучшение. loss теперь монотонно
 F1: 0.8005
 accuracy: 0.7292
Accuracy теперь выше уровня случайного угадывания, но все ещё хуже, чем у базово
```

In [27]:
```
my_log_reg = MyLogReg(learning_rate=0.0001, max_iter=1000, alpha=0.001)
my_log_reg.fit(X_train, Y_train)
Y_prediction_my = my_log_reg.predict(X_test)
f1_my_log = f1_score(Y_test, Y_prediction_my)
accuracy_my_log = accuracy_score(Y_test, Y_prediction_my)
print(f" F1: {f1_my_log:.4f}")
print(f" accuracy: {accuracy_my_log:.4f}")
```

```
iter 0, loss = 0.693147
iter 100, loss = 0.548904
iter 200, loss = 0.545975
iter 300, loss = 0.545288
iter 400, loss = 0.544822
iter 500, loss = 0.544440
iter 600, loss = 0.544115
iter 700, loss = 0.543832
iter 800, loss = 0.543579
iter 900, loss = 0.543348
 F1: 0.8005
 accuracy: 0.7292
```

In [ ]:
```
Реализуем перебор гиперпараметров для собственной модели на полиномиальных призн
каждого параметра.Сохраняется комбинация, давшая наибольший f1 (т.к. учитывает д
F1=0.8372, accuracy=0.7822
best params: {'lr': 0.0001, 'alpha': 0.01}, best f1 = 0.8372
Наблюдается улчшение обеих метрик (accuracy +7,3%, f1 +4,6%). Собственная моднль
```

In [28]:
```
learning_rates = [1e-5, 3e-5, 1e-4]
alphas = [0.1, 0.01, 0.001, 0.0001]
best_f1 = 0
best_params = {}
for lr in learning_rates:
    for alpha in alphas:
        print(f"\n lr={lr}, alpha={alpha}")
        model = MyLogReg(learning_rate=lr, max_iter=5000, alpha=alpha)
        model.fit(X_train_poly_scaled, Y_train)
        Y_pred = model.predict(X_test_poly_scaled)
        f1 = f1_score(Y_test, Y_pred)
        accuracy = accuracy_score(Y_test, Y_pred)
        print(f"F1={f1:.4f}, accuracy={accuracy:.4f}")
        if f1>best_f1:
            best_f1 = f1
```

```
                best_params={'lr':lr, 'alpha':alpha}
print(f" best params: {best_params}, best f1 = {best_f1:.4f}")
```

```
 lr=1e-05, alpha=0.1
iter 0, loss = 0.693147
iter 100, loss = 0.692414
iter 200, loss = 0.691685
iter 300, loss = 0.690961
iter 400, loss = 0.690241
iter 500, loss = 0.689525
iter 600, loss = 0.688814
iter 700, loss = 0.688107
iter 800, loss = 0.687405
iter 900, loss = 0.686707
iter 1000, loss = 0.686013
iter 1100, loss = 0.685323
iter 1200, loss = 0.684638
iter 1300, loss = 0.683956
iter 1400, loss = 0.683279
iter 1500, loss = 0.682606
iter 1600, loss = 0.681937
iter 1700, loss = 0.681271
iter 1800, loss = 0.680610
iter 1900, loss = 0.679953
iter 2000, loss = 0.679300
iter 2100, loss = 0.678650
iter 2200, loss = 0.678005
iter 2300, loss = 0.677363
iter 2400, loss = 0.676725
iter 2500, loss = 0.676091
iter 2600, loss = 0.675460
iter 2700, loss = 0.674833
iter 2800, loss = 0.674210
iter 2900, loss = 0.673591
iter 3000, loss = 0.672975
iter 3100, loss = 0.672362
iter 3200, loss = 0.671754
iter 3300, loss = 0.671148
iter 3400, loss = 0.670546
iter 3500, loss = 0.669948
iter 3600, loss = 0.669353
iter 3700, loss = 0.668761
iter 3800, loss = 0.668173
iter 3900, loss = 0.667588
iter 4000, loss = 0.667007
iter 4100, loss = 0.666428
iter 4200, loss = 0.665853
iter 4300, loss = 0.665281
iter 4400, loss = 0.664713
iter 4500, loss = 0.664147
iter 4600, loss = 0.663585
iter 4700, loss = 0.663025
iter 4800, loss = 0.662469
iter 4900, loss = 0.661916
F1=0.8312, accuracy=0.7732

 lr=1e-05, alpha=0.01
iter 0, loss = 0.693147
iter 100, loss = 0.692414
iter 200, loss = 0.691685
iter 300, loss = 0.690960
iter 400, loss = 0.690240
iter 500, loss = 0.689525
```

```
iter 600, loss = 0.688813
iter 700, loss = 0.688106
iter 800, loss = 0.687403
iter 900, loss = 0.686704
iter 1000, loss = 0.686010
iter 1100, loss = 0.685320
iter 1200, loss = 0.684633
iter 1300, loss = 0.683951
iter 1400, loss = 0.683273
iter 1500, loss = 0.682599
iter 1600, loss = 0.681929
iter 1700, loss = 0.681263
iter 1800, loss = 0.680601
iter 1900, loss = 0.679943
iter 2000, loss = 0.679288
iter 2100, loss = 0.678638
iter 2200, loss = 0.677991
iter 2300, loss = 0.677348
iter 2400, loss = 0.676709
iter 2500, loss = 0.676073
iter 2600, loss = 0.675441
iter 2700, loss = 0.674813
iter 2800, loss = 0.674189
iter 2900, loss = 0.673568
iter 3000, loss = 0.672950
iter 3100, loss = 0.672336
iter 3200, loss = 0.671726
iter 3300, loss = 0.671119
iter 3400, loss = 0.670515
iter 3500, loss = 0.669915
iter 3600, loss = 0.669318
iter 3700, loss = 0.668725
iter 3800, loss = 0.668135
iter 3900, loss = 0.667548
iter 4000, loss = 0.666965
iter 4100, loss = 0.666384
iter 4200, loss = 0.665807
iter 4300, loss = 0.665234
iter 4400, loss = 0.664663
iter 4500, loss = 0.664095
iter 4600, loss = 0.663531
iter 4700, loss = 0.662969
iter 4800, loss = 0.662411
iter 4900, loss = 0.661856
F1=0.8312, accuracy=0.7732

 lr=1e-05, alpha=0.001
iter 0, loss = 0.693147
iter 100, loss = 0.692414
iter 200, loss = 0.691685
iter 300, loss = 0.690960
iter 400, loss = 0.690240
iter 500, loss = 0.689524
iter 600, loss = 0.688813
iter 700, loss = 0.688106
iter 800, loss = 0.687403
iter 900, loss = 0.686704
iter 1000, loss = 0.686010
iter 1100, loss = 0.685319
iter 1200, loss = 0.684633
```

```
iter 1300, loss = 0.683951
iter 1400, loss = 0.683273
iter 1500, loss = 0.682598
iter 1600, loss = 0.681928
iter 1700, loss = 0.681262
iter 1800, loss = 0.680600
iter 1900, loss = 0.679942
iter 2000, loss = 0.679287
iter 2100, loss = 0.678636
iter 2200, loss = 0.677990
iter 2300, loss = 0.677346
iter 2400, loss = 0.676707
iter 2500, loss = 0.676071
iter 2600, loss = 0.675439
iter 2700, loss = 0.674811
iter 2800, loss = 0.674186
iter 2900, loss = 0.673565
iter 3000, loss = 0.672948
iter 3100, loss = 0.672334
iter 3200, loss = 0.671723
iter 3300, loss = 0.671116
iter 3400, loss = 0.670512
iter 3500, loss = 0.669912
iter 3600, loss = 0.669315
iter 3700, loss = 0.668721
iter 3800, loss = 0.668131
iter 3900, loss = 0.667544
iter 4000, loss = 0.666961
iter 4100, loss = 0.666380
iter 4200, loss = 0.665803
iter 4300, loss = 0.665229
iter 4400, loss = 0.664658
iter 4500, loss = 0.664090
iter 4600, loss = 0.663525
iter 4700, loss = 0.662964
iter 4800, loss = 0.662405
iter 4900, loss = 0.661850
F1=0.8312, accuracy=0.7732

 lr=1e-05, alpha=0.0001
iter 0, loss = 0.693147
iter 100, loss = 0.692414
iter 200, loss = 0.691685
iter 300, loss = 0.690960
iter 400, loss = 0.690240
iter 500, loss = 0.689524
iter 600, loss = 0.688813
iter 700, loss = 0.688106
iter 800, loss = 0.687403
iter 900, loss = 0.686704
iter 1000, loss = 0.686010
iter 1100, loss = 0.685319
iter 1200, loss = 0.684633
iter 1300, loss = 0.683951
iter 1400, loss = 0.683272
iter 1500, loss = 0.682598
iter 1600, loss = 0.681928
iter 1700, loss = 0.681262
iter 1800, loss = 0.680600
iter 1900, loss = 0.679941
```

```
iter 2000, loss = 0.679287
iter 2100, loss = 0.678636
iter 2200, loss = 0.677989
iter 2300, loss = 0.677346
iter 2400, loss = 0.676707
iter 2500, loss = 0.676071
iter 2600, loss = 0.675439
iter 2700, loss = 0.674811
iter 2800, loss = 0.674186
iter 2900, loss = 0.673565
iter 3000, loss = 0.672947
iter 3100, loss = 0.672333
iter 3200, loss = 0.671723
iter 3300, loss = 0.671116
iter 3400, loss = 0.670512
iter 3500, loss = 0.669912
iter 3600, loss = 0.669315
iter 3700, loss = 0.668721
iter 3800, loss = 0.668131
iter 3900, loss = 0.667544
iter 4000, loss = 0.666960
iter 4100, loss = 0.666380
iter 4200, loss = 0.665802
iter 4300, loss = 0.665228
iter 4400, loss = 0.664657
iter 4500, loss = 0.664090
iter 4600, loss = 0.663525
iter 4700, loss = 0.662963
iter 4800, loss = 0.662405
iter 4900, loss = 0.661849
F1=0.8312, accuracy=0.7732

 lr=3e-05, alpha=0.1
iter 0, loss = 0.693147
iter 100, loss = 0.690961
iter 200, loss = 0.688814
iter 300, loss = 0.686707
iter 400, loss = 0.684637
iter 500, loss = 0.682605
iter 600, loss = 0.680610
iter 700, loss = 0.678650
iter 800, loss = 0.676725
iter 900, loss = 0.674833
iter 1000, loss = 0.672974
iter 1100, loss = 0.671148
iter 1200, loss = 0.669352
iter 1300, loss = 0.667588
iter 1400, loss = 0.665853
iter 1500, loss = 0.664146
iter 1600, loss = 0.662468
iter 1700, loss = 0.660818
iter 1800, loss = 0.659194
iter 1900, loss = 0.657596
iter 2000, loss = 0.656024
iter 2100, loss = 0.654476
iter 2200, loss = 0.652953
iter 2300, loss = 0.651453
iter 2400, loss = 0.649976
iter 2500, loss = 0.648521
iter 2600, loss = 0.647088
```

```
iter 2700, loss = 0.645676
iter 2800, loss = 0.644285
iter 2900, loss = 0.642914
iter 3000, loss = 0.641562
iter 3100, loss = 0.640230
iter 3200, loss = 0.638917
iter 3300, loss = 0.637622
iter 3400, loss = 0.636344
iter 3500, loss = 0.635085
iter 3600, loss = 0.633842
iter 3700, loss = 0.632615
iter 3800, loss = 0.631405
iter 3900, loss = 0.630211
iter 4000, loss = 0.629033
iter 4100, loss = 0.627869
iter 4200, loss = 0.626721
iter 4300, loss = 0.625587
iter 4400, loss = 0.624467
iter 4500, loss = 0.623361
iter 4600, loss = 0.622268
iter 4700, loss = 0.621189
iter 4800, loss = 0.620123
iter 4900, loss = 0.619069
F1=0.8324, accuracy=0.7755

 lr=3e-05, alpha=0.01
iter 0, loss = 0.693147
iter 100, loss = 0.690960
iter 200, loss = 0.688813
iter 300, loss = 0.686704
iter 400, loss = 0.684633
iter 500, loss = 0.682599
iter 600, loss = 0.680600
iter 700, loss = 0.678637
iter 800, loss = 0.676708
iter 900, loss = 0.674813
iter 1000, loss = 0.672950
iter 1100, loss = 0.671118
iter 1200, loss = 0.669318
iter 1300, loss = 0.667548
iter 1400, loss = 0.665807
iter 1500, loss = 0.664095
iter 1600, loss = 0.662410
iter 1700, loss = 0.660753
iter 1800, loss = 0.659122
iter 1900, loss = 0.657518
iter 2000, loss = 0.655938
iter 2100, loss = 0.654383
iter 2200, loss = 0.652852
iter 2300, loss = 0.651344
iter 2400, loss = 0.649858
iter 2500, loss = 0.648395
iter 2600, loss = 0.646954
iter 2700, loss = 0.645533
iter 2800, loss = 0.644133
iter 2900, loss = 0.642753
iter 3000, loss = 0.641393
iter 3100, loss = 0.640051
iter 3200, loss = 0.638728
iter 3300, loss = 0.637424
```

```
iter 3400, loss = 0.636137
iter 3500, loss = 0.634867
iter 3600, loss = 0.633614
iter 3700, loss = 0.632378
iter 3800, loss = 0.631158
iter 3900, loss = 0.629953
iter 4000, loss = 0.628764
iter 4100, loss = 0.627590
iter 4200, loss = 0.626431
iter 4300, loss = 0.625286
iter 4400, loss = 0.624156
iter 4500, loss = 0.623039
iter 4600, loss = 0.621935
iter 4700, loss = 0.620845
iter 4800, loss = 0.619768
iter 4900, loss = 0.618703
F1=0.8324, accuracy=0.7755

 lr=3e-05, alpha=0.001
iter 0, loss = 0.693147
iter 100, loss = 0.690960
iter 200, loss = 0.688813
iter 300, loss = 0.686704
iter 400, loss = 0.684633
iter 500, loss = 0.682598
iter 600, loss = 0.680600
iter 700, loss = 0.678636
iter 800, loss = 0.676707
iter 900, loss = 0.674811
iter 1000, loss = 0.672947
iter 1100, loss = 0.671115
iter 1200, loss = 0.669314
iter 1300, loss = 0.667544
iter 1400, loss = 0.665802
iter 1500, loss = 0.664089
iter 1600, loss = 0.662404
iter 1700, loss = 0.660747
iter 1800, loss = 0.659115
iter 1900, loss = 0.657510
iter 2000, loss = 0.655929
iter 2100, loss = 0.654373
iter 2200, loss = 0.652841
iter 2300, loss = 0.651333
iter 2400, loss = 0.649847
iter 2500, loss = 0.648383
iter 2600, loss = 0.646940
iter 2700, loss = 0.645519
iter 2800, loss = 0.644118
iter 2900, loss = 0.642737
iter 3000, loss = 0.641375
iter 3100, loss = 0.640033
iter 3200, loss = 0.638709
iter 3300, loss = 0.637404
iter 3400, loss = 0.636116
iter 3500, loss = 0.634845
iter 3600, loss = 0.633591
iter 3700, loss = 0.632354
iter 3800, loss = 0.631133
iter 3900, loss = 0.629927
iter 4000, loss = 0.628737
```

```
iter 4100, loss = 0.627562
iter 4200, loss = 0.626402
iter 4300, loss = 0.625256
iter 4400, loss = 0.624125
iter 4500, loss = 0.623007
iter 4600, loss = 0.621902
iter 4700, loss = 0.620811
iter 4800, loss = 0.619732
iter 4900, loss = 0.618667
F1=0.8324, accuracy=0.7755

 lr=3e-05, alpha=0.0001
iter 0, loss = 0.693147
iter 100, loss = 0.690960
iter 200, loss = 0.688813
iter 300, loss = 0.686704
iter 400, loss = 0.684633
iter 500, loss = 0.682598
iter 600, loss = 0.680599
iter 700, loss = 0.678636
iter 800, loss = 0.676706
iter 900, loss = 0.674810
iter 1000, loss = 0.672947
iter 1100, loss = 0.671115
iter 1200, loss = 0.669314
iter 1300, loss = 0.667543
iter 1400, loss = 0.665802
iter 1500, loss = 0.664089
iter 1600, loss = 0.662404
iter 1700, loss = 0.660746
iter 1800, loss = 0.659115
iter 1900, loss = 0.657509
iter 2000, loss = 0.655929
iter 2100, loss = 0.654373
iter 2200, loss = 0.652840
iter 2300, loss = 0.651332
iter 2400, loss = 0.649845
iter 2500, loss = 0.648381
iter 2600, loss = 0.646939
iter 2700, loss = 0.645517
iter 2800, loss = 0.644116
iter 2900, loss = 0.642735
iter 3000, loss = 0.641374
iter 3100, loss = 0.640031
iter 3200, loss = 0.638707
iter 3300, loss = 0.637402
iter 3400, loss = 0.636114
iter 3500, loss = 0.634843
iter 3600, loss = 0.633589
iter 3700, loss = 0.632352
iter 3800, loss = 0.631130
iter 3900, loss = 0.629925
iter 4000, loss = 0.628735
iter 4100, loss = 0.627560
iter 4200, loss = 0.626399
iter 4300, loss = 0.625253
iter 4400, loss = 0.624121
iter 4500, loss = 0.623003
iter 4600, loss = 0.621899
iter 4700, loss = 0.620807
```

```
iter 4800, loss = 0.619729
iter 4900, loss = 0.618663
F1=0.8324, accuracy=0.7755

 lr=0.0001, alpha=0.1
iter 0, loss = 0.693147
iter 100, loss = 0.686012
iter 200, loss = 0.679298
iter 300, loss = 0.672972
iter 400, loss = 0.667004
iter 500, loss = 0.661362
iter 600, loss = 0.656021
iter 700, loss = 0.650954
iter 800, loss = 0.646141
iter 900, loss = 0.641559
iter 1000, loss = 0.637190
iter 1100, loss = 0.633018
iter 1200, loss = 0.629029
iter 1300, loss = 0.625207
iter 1400, loss = 0.621543
iter 1500, loss = 0.618024
iter 1600, loss = 0.614641
iter 1700, loss = 0.611384
iter 1800, loss = 0.608247
iter 1900, loss = 0.605222
iter 2000, loss = 0.602301
iter 2100, loss = 0.599480
iter 2200, loss = 0.596751
iter 2300, loss = 0.594111
iter 2400, loss = 0.591555
iter 2500, loss = 0.589078
iter 2600, loss = 0.586676
iter 2700, loss = 0.584346
iter 2800, loss = 0.582084
iter 2900, loss = 0.579887
iter 3000, loss = 0.577752
iter 3100, loss = 0.575677
iter 3200, loss = 0.573658
iter 3300, loss = 0.571693
iter 3400, loss = 0.569780
iter 3500, loss = 0.567917
iter 3600, loss = 0.566102
iter 3700, loss = 0.564333
iter 3800, loss = 0.562608
iter 3900, loss = 0.560925
iter 4000, loss = 0.559284
iter 4100, loss = 0.557681
iter 4200, loss = 0.556117
iter 4300, loss = 0.554589
iter 4400, loss = 0.553096
iter 4500, loss = 0.551638
iter 4600, loss = 0.550212
iter 4700, loss = 0.548819
iter 4800, loss = 0.547455
iter 4900, loss = 0.546122
F1=0.8371, accuracy=0.7818

 lr=0.0001, alpha=0.01
iter 0, loss = 0.693147
iter 100, loss = 0.686009
```

```
iter 200, loss = 0.679286
iter 300, loss = 0.672948
iter 400, loss = 0.666962
iter 500, loss = 0.661300
iter 600, loss = 0.655935
iter 700, loss = 0.650843
iter 800, loss = 0.646001
iter 900, loss = 0.641389
iter 1000, loss = 0.636989
iter 1100, loss = 0.632784
iter 1200, loss = 0.628760
iter 1300, loss = 0.624904
iter 1400, loss = 0.621203
iter 1500, loss = 0.617647
iter 1600, loss = 0.614226
iter 1700, loss = 0.610932
iter 1800, loss = 0.607755
iter 1900, loss = 0.604691
iter 2000, loss = 0.601730
iter 2100, loss = 0.598868
iter 2200, loss = 0.596100
iter 2300, loss = 0.593419
iter 2400, loss = 0.590822
iter 2500, loss = 0.588304
iter 2600, loss = 0.585861
iter 2700, loss = 0.583490
iter 2800, loss = 0.581187
iter 2900, loss = 0.578948
iter 3000, loss = 0.576772
iter 3100, loss = 0.574655
iter 3200, loss = 0.572595
iter 3300, loss = 0.570589
iter 3400, loss = 0.568635
iter 3500, loss = 0.566732
iter 3600, loss = 0.564876
iter 3700, loss = 0.563066
iter 3800, loss = 0.561301
iter 3900, loss = 0.559578
iter 4000, loss = 0.557896
iter 4100, loss = 0.556254
iter 4200, loss = 0.554650
iter 4300, loss = 0.553083
iter 4400, loss = 0.551552
iter 4500, loss = 0.550054
iter 4600, loss = 0.548590
iter 4700, loss = 0.547158
iter 4800, loss = 0.545757
iter 4900, loss = 0.544386
F1=0.8372, accuracy=0.7822

 lr=0.0001, alpha=0.001
iter 0, loss = 0.693147
iter 100, loss = 0.686009
iter 200, loss = 0.679285
iter 300, loss = 0.672945
iter 400, loss = 0.666958
iter 500, loss = 0.661294
iter 600, loss = 0.655926
iter 700, loss = 0.650832
iter 800, loss = 0.645987
```

```
iter 900, loss = 0.641372
iter 1000, loss = 0.636969
iter 1100, loss = 0.632761
iter 1200, loss = 0.628733
iter 1300, loss = 0.624873
iter 1400, loss = 0.621169
iter 1500, loss = 0.617609
iter 1600, loss = 0.614185
iter 1700, loss = 0.610886
iter 1800, loss = 0.607706
iter 1900, loss = 0.604637
iter 2000, loss = 0.601673
iter 2100, loss = 0.598807
iter 2200, loss = 0.596034
iter 2300, loss = 0.593350
iter 2400, loss = 0.590748
iter 2500, loss = 0.588226
iter 2600, loss = 0.585779
iter 2700, loss = 0.583404
iter 2800, loss = 0.581096
iter 2900, loss = 0.578854
iter 3000, loss = 0.576674
iter 3100, loss = 0.574552
iter 3200, loss = 0.572488
iter 3300, loss = 0.570478
iter 3400, loss = 0.568520
iter 3500, loss = 0.566612
iter 3600, loss = 0.564752
iter 3700, loss = 0.562939
iter 3800, loss = 0.561169
iter 3900, loss = 0.559442
iter 4000, loss = 0.557757
iter 4100, loss = 0.556111
iter 4200, loss = 0.554503
iter 4300, loss = 0.552932
iter 4400, loss = 0.551396
iter 4500, loss = 0.549895
iter 4600, loss = 0.548427
iter 4700, loss = 0.546991
iter 4800, loss = 0.545586
iter 4900, loss = 0.544211
F1=0.8372, accuracy=0.7822

 lr=0.0001, alpha=0.0001
iter 0, loss = 0.693147
iter 100, loss = 0.686009
iter 200, loss = 0.679285
iter 300, loss = 0.672945
iter 400, loss = 0.666957
iter 500, loss = 0.661293
iter 600, loss = 0.655925
iter 700, loss = 0.650830
iter 800, loss = 0.645985
iter 900, loss = 0.641370
iter 1000, loss = 0.636967
iter 1100, loss = 0.632758
iter 1200, loss = 0.628731
iter 1300, loss = 0.624870
iter 1400, loss = 0.621166
iter 1500, loss = 0.617605
```

```
iter 1600, loss = 0.614180
iter 1700, loss = 0.610882
iter 1800, loss = 0.607701
iter 1900, loss = 0.604632
iter 2000, loss = 0.601667
iter 2100, loss = 0.598801
iter 2200, loss = 0.596028
iter 2300, loss = 0.593343
iter 2400, loss = 0.590741
iter 2500, loss = 0.588218
iter 2600, loss = 0.585771
iter 2700, loss = 0.583395
iter 2800, loss = 0.581087
iter 2900, loss = 0.578844
iter 3000, loss = 0.576664
iter 3100, loss = 0.574542
iter 3200, loss = 0.572477
iter 3300, loss = 0.570467
iter 3400, loss = 0.568509
iter 3500, loss = 0.566600
iter 3600, loss = 0.564740
iter 3700, loss = 0.562926
iter 3800, loss = 0.561156
iter 3900, loss = 0.559429
iter 4000, loss = 0.557743
iter 4100, loss = 0.556096
iter 4200, loss = 0.554488
iter 4300, loss = 0.552916
iter 4400, loss = 0.551381
iter 4500, loss = 0.549879
iter 4600, loss = 0.548411
iter 4700, loss = 0.546974
iter 4800, loss = 0.545569
iter 4900, loss = 0.544194
F1=0.8372, accuracy=0.7822
 best params: {'lr': 0.0001, 'alpha': 0.01}, best f1 = 0.8372
```

In [ ]:
```
Обучение с увеличенным числом итераций. Используются найденные лучшие параметры
f1=0.8502, accuracy=0.7950
Наблюдается улучшение обеих метрик, текущие показатели свидетельствуют о том, чт
```

In [29]:
```
final_model= MyLogReg(learning_rate=best_params['lr'], max_iter=50000, alpha=bes
final_model.fit(X_train_poly_scaled, Y_train)
Y_pred_final = final_model.predict(X_test_poly_scaled)
f1_final = f1_score(Y_test, Y_pred_final)
accuracy_final = accuracy_score(Y_test, Y_pred_final)
print(f" f1={f1_final:.4f}, accuracy={accuracy_final:.4f}")
```

```
iter 0, loss = 0.693147
iter 100, loss = 0.686009
iter 200, loss = 0.679286
iter 300, loss = 0.672948
iter 400, loss = 0.666962
iter 500, loss = 0.661300
iter 600, loss = 0.655935
iter 700, loss = 0.650843
iter 800, loss = 0.646001
iter 900, loss = 0.641389
iter 1000, loss = 0.636989
iter 1100, loss = 0.632784
iter 1200, loss = 0.628760
iter 1300, loss = 0.624904
iter 1400, loss = 0.621203
iter 1500, loss = 0.617647
iter 1600, loss = 0.614226
iter 1700, loss = 0.610932
iter 1800, loss = 0.607755
iter 1900, loss = 0.604691
iter 2000, loss = 0.601730
iter 2100, loss = 0.598868
iter 2200, loss = 0.596100
iter 2300, loss = 0.593419
iter 2400, loss = 0.590822
iter 2500, loss = 0.588304
iter 2600, loss = 0.585861
iter 2700, loss = 0.583490
iter 2800, loss = 0.581187
iter 2900, loss = 0.578948
iter 3000, loss = 0.576772
iter 3100, loss = 0.574655
iter 3200, loss = 0.572595
iter 3300, loss = 0.570589
iter 3400, loss = 0.568635
iter 3500, loss = 0.566732
iter 3600, loss = 0.564876
iter 3700, loss = 0.563066
iter 3800, loss = 0.561301
iter 3900, loss = 0.559578
iter 4000, loss = 0.557896
iter 4100, loss = 0.556254
iter 4200, loss = 0.554650
iter 4300, loss = 0.553083
iter 4400, loss = 0.551552
iter 4500, loss = 0.550054
iter 4600, loss = 0.548590
iter 4700, loss = 0.547158
iter 4800, loss = 0.545757
iter 4900, loss = 0.544386
iter 5000, loss = 0.543044
iter 5100, loss = 0.541730
iter 5200, loss = 0.540444
iter 5300, loss = 0.539183
iter 5400, loss = 0.537949
iter 5500, loss = 0.536739
iter 5600, loss = 0.535553
iter 5700, loss = 0.534391
iter 5800, loss = 0.533251
iter 5900, loss = 0.532134
```

```
iter 6000, loss = 0.531037
iter 6100, loss = 0.529962
iter 6200, loss = 0.528907
iter 6300, loss = 0.527872
iter 6400, loss = 0.526855
iter 6500, loss = 0.525858
iter 6600, loss = 0.524878
iter 6700, loss = 0.523916
iter 6800, loss = 0.522972
iter 6900, loss = 0.522044
iter 7000, loss = 0.521132
iter 7100, loss = 0.520236
iter 7200, loss = 0.519356
iter 7300, loss = 0.518491
iter 7400, loss = 0.517640
iter 7500, loss = 0.516804
iter 7600, loss = 0.515982
iter 7700, loss = 0.515174
iter 7800, loss = 0.514378
iter 7900, loss = 0.513596
iter 8000, loss = 0.512827
iter 8100, loss = 0.512070
iter 8200, loss = 0.511325
iter 8300, loss = 0.510591
iter 8400, loss = 0.509870
iter 8500, loss = 0.509159
iter 8600, loss = 0.508460
iter 8700, loss = 0.507772
iter 8800, loss = 0.507093
iter 8900, loss = 0.506426
iter 9000, loss = 0.505768
iter 9100, loss = 0.505120
iter 9200, loss = 0.504482
iter 9300, loss = 0.503853
iter 9400, loss = 0.503234
iter 9500, loss = 0.502623
iter 9600, loss = 0.502021
iter 9700, loss = 0.501428
iter 9800, loss = 0.500844
iter 9900, loss = 0.500267
iter 10000, loss = 0.499699
iter 10100, loss = 0.499139
iter 10200, loss = 0.498586
iter 10300, loss = 0.498042
iter 10400, loss = 0.497504
iter 10500, loss = 0.496974
iter 10600, loss = 0.496451
iter 10700, loss = 0.495935
iter 10800, loss = 0.495426
iter 10900, loss = 0.494924
iter 11000, loss = 0.494428
iter 11100, loss = 0.493939
iter 11200, loss = 0.493457
iter 11300, loss = 0.492980
iter 11400, loss = 0.492510
iter 11500, loss = 0.492045
iter 11600, loss = 0.491587
iter 11700, loss = 0.491134
iter 11800, loss = 0.490687
iter 11900, loss = 0.490245
```

```
iter 12000, loss = 0.489809
iter 12100, loss = 0.489379
iter 12200, loss = 0.488953
iter 12300, loss = 0.488533
iter 12400, loss = 0.488118
iter 12500, loss = 0.487708
iter 12600, loss = 0.487303
iter 12700, loss = 0.486902
iter 12800, loss = 0.486507
iter 12900, loss = 0.486116
iter 13000, loss = 0.485729
iter 13100, loss = 0.485347
iter 13200, loss = 0.484969
iter 13300, loss = 0.484596
iter 13400, loss = 0.484227
iter 13500, loss = 0.483862
iter 13600, loss = 0.483501
iter 13700, loss = 0.483145
iter 13800, loss = 0.482792
iter 13900, loss = 0.482443
iter 14000, loss = 0.482098
iter 14100, loss = 0.481757
iter 14200, loss = 0.481419
iter 14300, loss = 0.481085
iter 14400, loss = 0.480755
iter 14500, loss = 0.480428
iter 14600, loss = 0.480105
iter 14700, loss = 0.479785
iter 14800, loss = 0.479468
iter 14900, loss = 0.479155
iter 15000, loss = 0.478845
iter 15100, loss = 0.478538
iter 15200, loss = 0.478235
iter 15300, loss = 0.477934
iter 15400, loss = 0.477637
iter 15500, loss = 0.477342
iter 15600, loss = 0.477051
iter 15700, loss = 0.476762
iter 15800, loss = 0.476476
iter 15900, loss = 0.476194
iter 16000, loss = 0.475913
iter 16100, loss = 0.475636
iter 16200, loss = 0.475361
iter 16300, loss = 0.475089
iter 16400, loss = 0.474820
iter 16500, loss = 0.474553
iter 16600, loss = 0.474289
iter 16700, loss = 0.474027
iter 16800, loss = 0.473767
iter 16900, loss = 0.473510
iter 17000, loss = 0.473256
iter 17100, loss = 0.473004
iter 17200, loss = 0.472754
iter 17300, loss = 0.472506
iter 17400, loss = 0.472261
iter 17500, loss = 0.472018
iter 17600, loss = 0.471777
iter 17700, loss = 0.471538
iter 17800, loss = 0.471302
iter 17900, loss = 0.471067
```

```
iter 18000, loss = 0.470835
iter 18100, loss = 0.470604
iter 18200, loss = 0.470376
iter 18300, loss = 0.470149
iter 18400, loss = 0.469925
iter 18500, loss = 0.469702
iter 18600, loss = 0.469482
iter 18700, loss = 0.469263
iter 18800, loss = 0.469046
iter 18900, loss = 0.468831
iter 19000, loss = 0.468617
iter 19100, loss = 0.468406
iter 19200, loss = 0.468196
iter 19300, loss = 0.467988
iter 19400, loss = 0.467782
iter 19500, loss = 0.467577
iter 19600, loss = 0.467374
iter 19700, loss = 0.467173
iter 19800, loss = 0.466973
iter 19900, loss = 0.466775
iter 20000, loss = 0.466578
iter 20100, loss = 0.466383
iter 20200, loss = 0.466190
iter 20300, loss = 0.465998
iter 20400, loss = 0.465807
iter 20500, loss = 0.465618
iter 20600, loss = 0.465431
iter 20700, loss = 0.465245
iter 20800, loss = 0.465060
iter 20900, loss = 0.464877
iter 21000, loss = 0.464695
iter 21100, loss = 0.464514
iter 21200, loss = 0.464335
iter 21300, loss = 0.464157
iter 21400, loss = 0.463981
iter 21500, loss = 0.463805
iter 21600, loss = 0.463632
iter 21700, loss = 0.463459
iter 21800, loss = 0.463287
iter 21900, loss = 0.463117
iter 22000, loss = 0.462948
iter 22100, loss = 0.462781
iter 22200, loss = 0.462614
iter 22300, loss = 0.462449
iter 22400, loss = 0.462284
iter 22500, loss = 0.462121
iter 22600, loss = 0.461959
iter 22700, loss = 0.461799
iter 22800, loss = 0.461639
iter 22900, loss = 0.461480
iter 23000, loss = 0.461323
iter 23100, loss = 0.461166
iter 23200, loss = 0.461011
iter 23300, loss = 0.460857
iter 23400, loss = 0.460703
iter 23500, loss = 0.460551
iter 23600, loss = 0.460400
iter 23700, loss = 0.460250
iter 23800, loss = 0.460100
iter 23900, loss = 0.459952
```

```
iter 24000, loss = 0.459805
iter 24100, loss = 0.459658
iter 24200, loss = 0.459513
iter 24300, loss = 0.459368
iter 24400, loss = 0.459225
iter 24500, loss = 0.459082
iter 24600, loss = 0.458941
iter 24700, loss = 0.458800
iter 24800, loss = 0.458660
iter 24900, loss = 0.458521
iter 25000, loss = 0.458382
iter 25100, loss = 0.458245
iter 25200, loss = 0.458109
iter 25300, loss = 0.457973
iter 25400, loss = 0.457838
iter 25500, loss = 0.457704
iter 25600, loss = 0.457571
iter 25700, loss = 0.457438
iter 25800, loss = 0.457307
iter 25900, loss = 0.457176
iter 26000, loss = 0.457046
iter 26100, loss = 0.456917
iter 26200, loss = 0.456788
iter 26300, loss = 0.456660
iter 26400, loss = 0.456533
iter 26500, loss = 0.456407
iter 26600, loss = 0.456281
iter 26700, loss = 0.456157
iter 26800, loss = 0.456032
iter 26900, loss = 0.455909
iter 27000, loss = 0.455786
iter 27100, loss = 0.455664
iter 27200, loss = 0.455543
iter 27300, loss = 0.455422
iter 27400, loss = 0.455302
iter 27500, loss = 0.455183
iter 27600, loss = 0.455064
iter 27700, loss = 0.454946
iter 27800, loss = 0.454829
iter 27900, loss = 0.454712
iter 28000, loss = 0.454596
iter 28100, loss = 0.454481
iter 28200, loss = 0.454366
iter 28300, loss = 0.454252
iter 28400, loss = 0.454138
iter 28500, loss = 0.454025
iter 28600, loss = 0.453913
iter 28700, loss = 0.453801
iter 28800, loss = 0.453690
iter 28900, loss = 0.453579
iter 29000, loss = 0.453469
iter 29100, loss = 0.453359
iter 29200, loss = 0.453250
iter 29300, loss = 0.453142
iter 29400, loss = 0.453034
iter 29500, loss = 0.452927
iter 29600, loss = 0.452820
iter 29700, loss = 0.452714
iter 29800, loss = 0.452608
iter 29900, loss = 0.452503
```

```
iter 30000, loss = 0.452399
iter 30100, loss = 0.452295
iter 30200, loss = 0.452191
iter 30300, loss = 0.452088
iter 30400, loss = 0.451985
iter 30500, loss = 0.451883
iter 30600, loss = 0.451782
iter 30700, loss = 0.451681
iter 30800, loss = 0.451580
iter 30900, loss = 0.451480
iter 31000, loss = 0.451380
iter 31100, loss = 0.451281
iter 31200, loss = 0.451182
iter 31300, loss = 0.451084
iter 31400, loss = 0.450987
iter 31500, loss = 0.450889
iter 31600, loss = 0.450792
iter 31700, loss = 0.450696
iter 31800, loss = 0.450600
iter 31900, loss = 0.450505
iter 32000, loss = 0.450410
iter 32100, loss = 0.450315
iter 32200, loss = 0.450221
iter 32300, loss = 0.450127
iter 32400, loss = 0.450034
iter 32500, loss = 0.449941
iter 32600, loss = 0.449848
iter 32700, loss = 0.449756
iter 32800, loss = 0.449664
iter 32900, loss = 0.449573
iter 33000, loss = 0.449482
iter 33100, loss = 0.449392
iter 33200, loss = 0.449302
iter 33300, loss = 0.449212
iter 33400, loss = 0.449123
iter 33500, loss = 0.449034
iter 33600, loss = 0.448945
iter 33700, loss = 0.448857
iter 33800, loss = 0.448769
iter 33900, loss = 0.448682
iter 34000, loss = 0.448595
iter 34100, loss = 0.448508
iter 34200, loss = 0.448422
iter 34300, loss = 0.448336
iter 34400, loss = 0.448250
iter 34500, loss = 0.448165
iter 34600, loss = 0.448080
iter 34700, loss = 0.447996
iter 34800, loss = 0.447912
iter 34900, loss = 0.447828
iter 35000, loss = 0.447744
iter 35100, loss = 0.447661
iter 35200, loss = 0.447578
iter 35300, loss = 0.447496
iter 35400, loss = 0.447414
iter 35500, loss = 0.447332
iter 35600, loss = 0.447251
iter 35700, loss = 0.447169
iter 35800, loss = 0.447089
iter 35900, loss = 0.447008
```

```
iter 36000, loss = 0.446928
iter 36100, loss = 0.446848
iter 36200, loss = 0.446769
iter 36300, loss = 0.446689
iter 36400, loss = 0.446610
iter 36500, loss = 0.446532
iter 36600, loss = 0.446453
iter 36700, loss = 0.446375
iter 36800, loss = 0.446298
iter 36900, loss = 0.446220
iter 37000, loss = 0.446143
iter 37100, loss = 0.446066
iter 37200, loss = 0.445990
iter 37300, loss = 0.445913
iter 37400, loss = 0.445837
iter 37500, loss = 0.445762
iter 37600, loss = 0.445686
iter 37700, loss = 0.445611
iter 37800, loss = 0.445536
iter 37900, loss = 0.445462
iter 38000, loss = 0.445387
iter 38100, loss = 0.445313
iter 38200, loss = 0.445240
iter 38300, loss = 0.445166
iter 38400, loss = 0.445093
iter 38500, loss = 0.445020
iter 38600, loss = 0.444947
iter 38700, loss = 0.444875
iter 38800, loss = 0.444803
iter 38900, loss = 0.444731
iter 39000, loss = 0.444659
iter 39100, loss = 0.444588
iter 39200, loss = 0.444516
iter 39300, loss = 0.444446
iter 39400, loss = 0.444375
iter 39500, loss = 0.444304
iter 39600, loss = 0.444234
iter 39700, loss = 0.444164
iter 39800, loss = 0.444095
iter 39900, loss = 0.444025
iter 40000, loss = 0.443956
iter 40100, loss = 0.443887
iter 40200, loss = 0.443818
iter 40300, loss = 0.443750
iter 40400, loss = 0.443682
iter 40500, loss = 0.443613
iter 40600, loss = 0.443546
iter 40700, loss = 0.443478
iter 40800, loss = 0.443411
iter 40900, loss = 0.443344
iter 41000, loss = 0.443277
iter 41100, loss = 0.443210
iter 41200, loss = 0.443144
iter 41300, loss = 0.443077
iter 41400, loss = 0.443011
iter 41500, loss = 0.442945
iter 41600, loss = 0.442880
iter 41700, loss = 0.442815
iter 41800, loss = 0.442749
iter 41900, loss = 0.442684
```

```
iter 42000, loss = 0.442620
iter 42100, loss = 0.442555
iter 42200, loss = 0.442491
iter 42300, loss = 0.442427
iter 42400, loss = 0.442363
iter 42500, loss = 0.442299
iter 42600, loss = 0.442236
iter 42700, loss = 0.442172
iter 42800, loss = 0.442109
iter 42900, loss = 0.442046
iter 43000, loss = 0.441983
iter 43100, loss = 0.441921
iter 43200, loss = 0.441859
iter 43300, loss = 0.441797
iter 43400, loss = 0.441735
iter 43500, loss = 0.441673
iter 43600, loss = 0.441611
iter 43700, loss = 0.441550
iter 43800, loss = 0.441489
iter 43900, loss = 0.441428
iter 44000, loss = 0.441367
iter 44100, loss = 0.441306
iter 44200, loss = 0.441246
iter 44300, loss = 0.441186
iter 44400, loss = 0.441126
iter 44500, loss = 0.441066
iter 44600, loss = 0.441006
iter 44700, loss = 0.440946
iter 44800, loss = 0.440887
iter 44900, loss = 0.440828
iter 45000, loss = 0.440769
iter 45100, loss = 0.440710
iter 45200, loss = 0.440651
iter 45300, loss = 0.440593
iter 45400, loss = 0.440534
iter 45500, loss = 0.440476
iter 45600, loss = 0.440418
iter 45700, loss = 0.440361
iter 45800, loss = 0.440303
iter 45900, loss = 0.440245
iter 46000, loss = 0.440188
iter 46100, loss = 0.440131
iter 46200, loss = 0.440074
iter 46300, loss = 0.440017
iter 46400, loss = 0.439960
iter 46500, loss = 0.439904
iter 46600, loss = 0.439848
iter 46700, loss = 0.439791
iter 46800, loss = 0.439735
iter 46900, loss = 0.439680
iter 47000, loss = 0.439624
iter 47100, loss = 0.439568
iter 47200, loss = 0.439513
iter 47300, loss = 0.439458
iter 47400, loss = 0.439403
iter 47500, loss = 0.439348
iter 47600, loss = 0.439293
iter 47700, loss = 0.439238
iter 47800, loss = 0.439184
iter 47900, loss = 0.439129
```

```
iter 48000, loss = 0.439075
iter 48100, loss = 0.439021
iter 48200, loss = 0.438967
iter 48300, loss = 0.438913
iter 48400, loss = 0.438860
iter 48500, loss = 0.438806
iter 48600, loss = 0.438753
iter 48700, loss = 0.438700
iter 48800, loss = 0.438647
iter 48900, loss = 0.438594
iter 49000, loss = 0.438541
iter 49100, loss = 0.438489
iter 49200, loss = 0.438436
iter 49300, loss = 0.438384
iter 49400, loss = 0.438332
iter 49500, loss = 0.438280
iter 49600, loss = 0.438228
iter 49700, loss = 0.438176
iter 49800, loss = 0.438124
iter 49900, loss = 0.438073
 f1=0.8502, accuracy=0.7950
```

In [ ]:

```
## Выводы:
Первоначальная модель с параметрами по умолчанию, обученная на сырых признаках,
метрики остались на том же уровне. Добавление полиномиальных признаков второй ст
взаимодействия, это привело к заметному росту качества: accuracy=0.8465, f1=0.88
гиперпараметров, результат на лучшей комбинации accuracy=0.8606. Реализована соб
результаты (F1: 0.7939 accuracy: 0.6717), было выявлено критическое значение мас
alpha стабилизировал обучение. Хотя итоговые метрики (f1=0.8502, accuracy=0.7950
модели и её способность обучаться.
```